Parsek — Timeline System Design

Comprehensive design specification for Parsek's unified timeline view — a chronological, read-only query layer across recordings and game actions that gives the player a single place to see everything that has happened and will happen on the committed timeline.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones. This document specifies how the timeline aggregates and presents data from the flight recorder system (see `parsek-flight-recorder-design.md`) and the game actions system (see `parsek-game-actions-and-resources-recorder-design.md`).

Version: 0.1 (Design) **Status:** Design phase. Depends on v0.6 (Game Actions System) being complete. **Out of scope:** Recording, playback, ghost visuals, vessel spawning. See `parsek-flight-recorder-design.md` . Resource tracking, ledger, recalculation engine. See `parsek-game-actions-and-resources-recorder-design.md` . Vessel-level telemetry (part events, segment events) — these remain in the Recordings Manager.

---

1. Introduction

Parsek has two existing views of committed data:

- The **Recordings Manager** is vessel-centric: a list of recordings with per-recording controls (enable/disable, rewind, loop, group). It answers "what vessels did I record?"
- The **Game Actions window** is resource-centric: a flat list of ledger actions and legacy game state events with sorting and deletion. It answers "what resource transactions happened?"

Neither answers the question the player actually has: **what happened, and what will happen, in chronological order across all systems?**

The timeline is a unified, chronological view that pulls from recordings and game actions, normalizes them into a common entry shape, and presents them sorted by universal time (UT). It replaces the Game Actions window entirely.

1.1 What the timeline is

The timeline is a **read-only query layer**. It does not own data. Recordings remain in `RecordingStore` . Game actions remain in the `Ledger` . The timeline reads from both, constructs a flat list of entries, and presents it.

This is analogous to a database view: it computes a result set from underlying tables. If a recording is committed or rewound, the timeline rebuilds. If a ledger action is added (facility upgrade, tech unlock), the timeline rebuilds. The timeline never modifies the data it reads.

1.2 What the timeline shows — and what it doesn't

The timeline shows **career-level events**: when missions launched and ended, what milestones were achieved, what resources changed, when contracts completed, when the KSC was upgraded. These are the events that shape the player's career progression.

The timeline does **not** show vessel-level telemetry: engine ignitions, staging events, parachute deployments, solar panel toggles, RCS pulses. That detail belongs in the Recordings Manager, where the player manages individual recordings. A decoupler firing or an engine throttle change is not a career event — it's a flight detail.

The dividing line: **if it changed a resource, a contract, a milestone, a kerbal, or a facility — it's in the timeline. If it only changed a vessel's physical state — it's in the Recordings Manager.**

1.3 What the player sees

The player sees the full committed history at all times. The timeline has a **current-UT marker** that divides the list into two visual halves:

- **Above the marker** (past): events that have already played out. Vessels spawned, resources applied, milestones credited. Full color.
- **Below the marker** (future): events that will play out when the player advances time. Ghost vessels will appear, resources will change, milestones will trigger. Dimmed but not hidden.

There are no branches and no hidden future — the player recorded that future and committed it. The timeline shows the consequences of their choices.

Situation	What the player sees in the timeline
Three recordings committed	All three recordings' career events interleaved chronologically — launches, recoveries, milestones, science, contracts, all sorted by UT
Rewind to before a recording	That recording's events appear below the current-UT marker (future), dimmed. They will play out again as the player advances.
Active flight in progress	Uncommitted events are not shown — they join the timeline on commit. The timeline shows only committed history.
Facility upgrade at KSC	The upgrade action appears instantly in the timeline at the current UT.
Contract completed during a recording	The completion action appears at the UT it occurred, interleaved with other career events at similar UTs.

1.4 How the timeline differs from the Recordings Manager

The Recordings Manager and the timeline are **complementary views of the same data**, cross-linked but not merged:

Aspect	Recordings Manager	Timeline
Organized by	Vessel (one row per recording)	Time (one row per event)
Shows	Recording metadata, playback controls, chain/loop config, part events	Career events from all systems, interleaved chronologically
Controls	Enable/disable, rewind, loop, group, hide	Rewind (same mechanism), filter
Answers	"What recordings do I have? What did this vessel do?"	"What happened at UT 5000? How did my career progress?"
Cross-link	Click recording → timeline scrolls to it	Click timeline recording entry → Recordings Manager selects it

Both windows can be open simultaneously.

1.5 Two data sources

The timeline draws from two systems. Each system owns its data; the timeline only reads.

1. Recordings (`RecordingStore.CommittedRecordings`)

Each committed recording contributes:

- A **launch event** at `rec.StartUT` (vessel name)
- A **terminal event** at `rec.EndUT` (recovering, destroyed, orbiting, stranded)
- A **spawn event** at `rec.StartUT` if the recording produces a ghost vessel
- A **ghost chain window** — a single row at the chain's start UT showing the full ghost duration: "Vessel Name: ghost UT 500–1600". Computed from chain members' UT ranges.

Recordings do **not** contribute part events, segment events, or flag events to the timeline. Those remain in the Recordings Manager.

2. Game actions (`Ledger.Actions`)

Each ledger action is a single timeline entry: science earned, funds spent, milestone achieved, contract completed, kerbal hired, facility upgraded, strategy activated. The ledger is the canonical source — the timeline uses `GameActionDisplay` for descriptions and colors, and reads derived fields (`Effective` , `Affordable` , `EffectiveScience` , etc.) set by the recalculation engine.

Game actions that occurred during a recording carry a `RecordingId` linking them to the recording. This connection is displayed as metadata (vessel name tag) but the entries remain at root level in the chronological list — they are career events, not vessel events.

Legacy events: saves started before the ledger system have committed events stored as `GameStateEvent` entries in `MilestoneStore` . The timeline displays these in their existing format using `GameStateEventDisplay` . As saves migrate to the ledger system, these disappear. Uncommitted events from the current flight session are **not** shown.

---

2. Design Philosophy

2.1 Read-only, not read-write

The timeline displays data. It never modifies it. The one action it offers — rewind — is an operation on the game state (loading a quicksave), not on the timeline's data.

Why: the timeline pulls from independent systems with their own persistence, consistency rules, and mutation APIs. Allowing edits through the timeline would require the timeline to own or mutate data it doesn't own, creating synchronization complexity.

2.2 Time-centric, not vessel-centric or resource-centric

The Recordings Manager groups by vessel. The Game Actions window groups by resource type. The timeline groups by **time**. This reveals causal chains that neither other view shows:

> *UT 600: Milestone "First orbit (Kerbin)" → +5000 funds UT 800: Facility upgrade "Runway → Lv.2" → -18000 funds UT 900: Contract complete "Orbit Kerbin" → +8000 funds*

The player sees that the milestone funded the runway upgrade. This causal chain is invisible in the Recordings Manager (which shows the recording as a single block) and invisible in the Game Actions window (which sorts by type, not time, by default).

2.3 Career events, not vessel telemetry

The timeline shows things that change the player's career state: resources, contracts, milestones, kerbals, facilities, strategies. It does not show what a vessel did physically — that's the Recordings Manager's job.

This keeps the timeline focused. A player with 12 recordings doesn't want to scroll through hundreds of engine throttle changes and solar panel toggles to find the contract completion that funded their next mission.

2.4 Full visibility, no hidden future

Both past and future events are always visible. Future events are visually dimmed but never hidden. This is consistent with Parsek's core principle: the player recorded that future and committed it. They should see the consequences of their commitments.

2.5 Significance filtering

The timeline can contain many events for an active career. Rather than hiding events, the timeline classifies them into **significance tiers** and defaults to showing only the high-level skeleton ("Overview"). The player can switch to "Detail" to see all resource transactions. This keeps the default view clean without hiding information.

2.6 The timeline replaces the Game Actions window

The timeline subsumes everything the Game Actions window does (showing game actions sorted by time) and adds recording lifecycle events and visual structure. Keeping both would create redundancy. The Game Actions button becomes the Timeline button. The resource budget summary, epoch display, and retired kerbals section move into the timeline window.

The current Game Actions window lives in `UI/ActionsWindowUI.cs`, one of six extracted UI windows (`RecordingsTableUI`, `SettingsWindowUI`, `TestRunnerUI`, `GroupPickerUI`, `SpawnControlUI`). The timeline replaces `ActionsWindowUI` with `UI/TimelineWindowUI.cs`, following the same extracted-window pattern: constructor takes `ParsekUI` parent, `DrawIfOpen(Rect)` entry point, `IsOpen` property, `ReleaseInputLock()` for cleanup.

---

3. Architecture

3.1 Coupling to other systems

The timeline has **read-only coupling** to two systems (plus legacy):

```
RecordingStore ──reads──→ TimelineBuilder ←──reads── Ledger
                              ↑
                          reads│ (legacy saves only)
                              │
                         MilestoneStore
```

The timeline never writes to any of these systems. The only side effect it triggers is through the rewind button, which delegates to `RecordingStore.InitiateRewind()` via `RecordingsTableUI.ShowRewindConfirmation()` — the same mechanism already used by the Recordings Manager.

3.2 TimelineEntry — the normalized event shape

Every event from every source is normalized into a common shape:

```
TimelineEntry
    double            UT              — universal time
    TimelineEntryType Type            — discriminator enum (see §3.3)
    string            DisplayText     — one-line human-readable description
    TimelineSource    Source          — Recording, GameAction, Legacy, or Derived
```

```
    SignificanceTier  Tier            — T1 or T2
    Color             DisplayColor    — green (earning), red (spending), white (neutral)
    string            RecordingId     — source recording ID; null for KSC-only actions
    string            VesselName      — vessel name if applicable; null otherwise
    bool              IsEffective     — false if zeroed by recalculation (duplicate milestone,
etc.)
```

`TimelineEntry` is a **view object** — constructed on demand, never serialized, never persisted.

3.3 TimelineEntryType

Discriminator enum covering every event source:

Recording lifecycle (3 types): `RecordingStart` , `RecordingEnd` , `VesselSpawn`

Game actions (23 types, 1:1 with `GameActionType`): `ScienceEarning` , `ScienceSpending` , `FundsEarning` , `FundsSpending` , `ReputationEarning` , `ReputationPenalty` , `MilestoneAchievement` , `ContractAccept` , `ContractComplete` , `ContractFail` , `ContractCancel` , `KerbalAssignment` , `KerbalHire` , `KerbalRescue` , `KerbalStandIn` , `FacilityUpgrade` , `FacilityDestruction` , `FacilityRepair` , `StrategyActivate` , `StrategyDeactivate` , `FundsInitial` , `ScienceInitial` , `ReputationInitial`

Derived (1 type): `GhostChainWindow` — a duration entry showing when a vessel is ghosted

Legacy (1 type): `LegacyEvent` — for `GameStateEvent` entries from pre-ledger `MilestoneStore` milestones

Total: 28 types.

3.4 Entry collection

`TimelineBuilder.Build()` constructs the entry list from source data. It accepts data sources as parameters (not reading global state directly) for testability:

```
TimelineBuilder.Build(
    IReadOnlyList<Recording> committedRecordings,
    IReadOnlyList<GameAction> ledgerActions,
    IReadOnlyList<Milestone> milestones,      // legacy
    uint currentEpoch,
    double currentUT
) → List<TimelineEntry>
```

Three collectors run in sequence, then results are merged:

Recording Collector — iterates committed recordings:

- Emits `RecordingStart` at `rec.StartUT` using vessel name
- Emits `RecordingEnd` at `rec.EndUT` using `rec.TerminalStateValue` for display text (e.g., "Recovering", "Destroyed", "Orbiting Kerbin")
- Emits `VesselSpawn` at `rec.StartUT` if `rec.PlaybackEnabled` and recording will produce a ghost
- Computes ghost chain windows: for each distinct `ChainId` among committed recordings (branch 0 only), finds the earliest `StartUT` and latest `EndUT` across all chain members, emits a `GhostChainWindow` entry with display text "VesselName: ghost UT start–end"
- Skips hidden recordings (`rec.Hidden`) unless a "show hidden" flag is set

Game Action Collector — iterates `Ledger.Actions` :

- Maps each `GameAction.Type` directly to the matching `TimelineEntryType`
- Uses `GameActionDisplay.GetDescription(action)` for display text
- Uses `GameActionDisplay.GetColor(action.Type)` for display color
- Reads `action.Effective` into `entry.IsEffective`
- Reads `action.RecordingId` into `entry.RecordingId`
- For `ScienceInitial` and `ReputationInitial` (which have no case in `GameActionDisplay.GetDescription`), provides custom descriptions: "Starting science: {value}", "Starting reputation: {value}"

Legacy Collector — iterates `MilestoneStore.Milestones` for the current epoch:

- For each committed milestone matching `currentEpoch`, iterates its events
- Skips resource events and crew status changes (`IsMilestoneFilteredEvent`)
- Uses `GameStateEventDisplay.GetDisplayCategory` and `GetDisplayDescription` for display text
- All legacy entries are T2

Merge — concatenate all entries from all three collectors, stable sort by UT ascending.

3.5 Cache invalidation

The entry list only changes on explicit user actions:

- Recording committed → rebuild
- Rewind completed → rebuild
- KSC spending (tech unlock, facility upgrade, kerbal hire) → rebuild
- Scene change → rebuild
- Warp exit (`onTimeWarpRateChanged` when rate drops to 1) → rebuild

The current-UT divider position updates on every draw call (reads `Planetarium.GetUniversalTime()`, finds insertion index via binary search in the cached sorted list — cheap). The entry list itself is only rebuilt on the triggers above.

3.6 TimelineSource

```
enum TimelineSource
{
    Recording,      // RecordingStore — recording lifecycle events
    GameAction,     // Ledger — resource transactions, contracts, milestones, kerbals, etc.
    Legacy,         // MilestoneStore — pre-ledger game state events
    Derived,        // Computed by timeline builder (ghost chain windows)
}
```

---

4. Significance Tiers

With part events excluded from the timeline, two tiers are sufficient: **Overview** (mission structure) and **Detail** (all resource transactions).

4.1 T1 — Overview (default)

High-level mission structure. Shows the skeleton of the player's career.

Entry Type	Why T1

RecordingStart	Mission launch — fundamental timeline anchor
RecordingEnd	Mission conclusion — the outcome matters
VesselSpawn	Ghost materialization — directly observable in the world
MilestoneAchievement	One-time progression gate, often with large rewards
ContractComplete	Mission objective achieved — major career event
ContractFail	Mission objective failed — consequences visible
FacilityUpgrade	KSC progression — enables new capabilities
FacilityDestruction	KSC regression — disables capabilities
KerbalHire	Roster expansion — funds spent, new capability
GhostChainWindow	Explains why a vessel is untouchable and when it resolves
FundsInitial	Career start — baseline reference
ScienceInitial	Career start — baseline reference
ReputationInitial	Career start — baseline reference

4.2 T2 — Detail

All resource transactions and supporting events. Visible when the player switches to "Detail".

Entry Type	Why T2
ScienceEarning	Experiment credit — progression
ScienceSpending	Tech unlock — progression
FundsEarning	Income — contract reward, recovery, milestone bonus
FundsSpending	Expense — vessel build, facility, hire
ReputationEarning	Rep gain
ReputationPenalty	Rep loss
ContractAccept	Commitment — advance payment, slot consumed
ContractCancel	Commitment withdrawn — penalties applied
KerbalAssignment	Crew mission tracking
KerbalRescue	Crew recovery — roster change
KerbalStandIn	Replacement crew — explains roster changes
FacilityRepair	KSC restoration
StrategyActivate	Policy change — reward diversion begins

StrategyDeactivate	Policy change — diversion ends
LegacyEvent	Pre-ledger committed events

4.3 Visibility rules

- **Overview** (default): T1 only. Clean career skeleton.
- **Detail**: T1 + T2. All career events including resource transactions.
- **Ineffective entries** (`IsEffective == false`): demoted one tier (T1→T2). A duplicate milestone that would normally be T1 only appears at Detail level. Rendered with strikethrough styling and gray color. This keeps the Overview clean — only things that actually mattered.

---

5. Player Interaction

This section describes what the player sees and does at the UI level.

5.1 Opening the timeline

The main Parsek window's "Game Actions" button becomes "Timeline". The badge count shows `MilestoneStore.GetPendingEventCount() + GameStateStore.GetUncommittedEventCount()` (same as today). Clicking it opens the timeline sub-window.

5.2 Window layout

The timeline window has four visual zones, top to bottom:

Zone 1: Resource Budget — identical to the current Game Actions window's resource budget summary. Shows reserved funds, science, and reputation versus available amounts. Over-committed resources highlighted in red with warning text. In sandbox mode, this zone is hidden (no resources). In science mode, only science is shown.

Zone 2: Filter Bar — a row of controls:

- **Tier selector**: two buttons labeled "Overview" (default) and "Detail". Overview shows mission structure — launches, recoveries, milestones, contracts, facility changes. Detail adds all resource transactions.
- **Source toggles**: "Recordings" and "Actions" toggle buttons. Both on by default. Turning off "Recordings" hides recording lifecycle events. Turning off "Actions" hides game action entries.

Zone 3: Entry List — the main scrollable list of timeline entries, divided by the current-UT marker.

Zone 4: Footer — epoch display ("Epoch: N (N reverts)"), retired kerbals count, and Close button.

5.3 The entry list

A flat, chronological list. No grouping, no collapse, no indentation. Every entry is a row at root level at its UT.

```
UT 500   Launch: Mun Explorer                          [↺]
UT 600   Milestone: First orbit (Kerbin) +5000 funds
UT 610   Contract complete: Orbit Kerbin +8000 funds
UT 800   Facility upgrade: Runway → Lv.2 -18000
UT 900   Science: crewReport@MunSrfLanded +8.0 sci
UT 3200  Recovery: Mun Explorer +12000 funds

──────── UT 3500 (now) ──────────────────────────
```

```
UT 4000  Launch: Station Core                          [↺]    ← dimmed
UT 4100  Contract complete: Build Station +25000       ← dimmed
UT 6000  Recovery: Station Core +8000 funds            ← dimmed
```

Each entry row contains:

- **UT column** (fixed width ~90px) — formatted via `KSPUtil.PrintDateCompact(ut, true)`
- **Color indicator** — small colored label matching the entry's `DisplayColor`: green for earnings, red for spendings, white for neutral
- **Description** — the `DisplayText` string, filling remaining width
- **Rewind button** [↺] — only on `RecordingStart` entries

Game actions that occurred during a recording show the vessel name in their description where useful (e.g., "Recovery: Mun Explorer +12000 funds" rather than just "Recovery +12000 funds"), since the `RecordingId` links to a vessel name.

Past events (above NOW divider): full color. **Future events** (below NOW divider): 50% alpha, dimmed but visible.

5.4 Current-UT divider

A horizontal separator line with "── UT {value} (now) ──" label. Updates every draw call. Does **not** live-update during time warp — jumps on warp exit.

5.5 Rewind from the timeline

Clicking the [↺] button on a `RecordingStart` entry invokes the same rewind flow as the Recordings Manager:

1. `RecordingsTableUI.ShowRewindConfirmation(rec)` displays a confirmation dialog: "Rewind to 'Vessel Name' launch at UT?"
2. The dialog shows how many future recordings will replay as ghosts
3. On confirm, `RecordingStore.InitiateRewind(rec)` executes the rewind

No new rewind infrastructure is needed.

5.6 Ineffective entries

Game action entries where `Effective = false` (duplicate milestones, duplicate contract completions) are:

- Demoted one tier (T1→T2) — invisible in Overview, visible in Detail
- Rendered with strikethrough text and gray color
- Suffixed with "(duplicate)" or "(unaffordable)"

5.7 Cross-link with Recordings Manager

The timeline and Recordings Manager share a `selectedRecordingId`:

- **Timeline → Manager**: clicking a `RecordingStart` entry sets `selectedRecordingId`. The Recordings Manager scrolls to and highlights that recording.
- **Manager → Timeline**: selecting a recording in the Recordings Manager sets `selectedRecordingId`. The timeline scrolls to that recording's launch entry.

Implementation note: IMGUI scroll views have no built-in "scroll to element" API. The implementation computes Y offset via row-height accumulation during the draw pass.

---

6. Game Mode Considerations

6.1 Career mode

All features active. All three resource sparklines available. All game action types appear.

6.2 Science mode

- No funds, no reputation, no contracts, no strategies
- Resource budget shows only science
- Sparkline shows only science
- Fund/reputation/contract/strategy entries never appear

6.3 Sandbox mode

- No game actions (empty ledger)
- Resource budget hidden, sparkline hidden
- Timeline shows only recording lifecycle events (launches, recoveries, spawns, ghost chains)
- Still useful: chronological mission structure

---

7. What the Timeline Does Not Do

Explicit non-goals for Phase 9:

- **Vessel-level telemetry**: part events (staging, engines, parachutes, docking, deployables, RCS, robotics), segment events, flag events. These belong in the Recordings Manager.
- **Resource sparkline / graphs**: deferred. May be added later based on player needs.
- **Individual event deletion**: the timeline is read-only. Delete buttons from the old Actions window are removed.
- **Sorting by columns**: always sorted by UT. Filtering (not sorting) finds specific event types.
- **Live UT marker during warp**: jumps on warp exit. Live-updating is a potential future optimization.
- **"Jump to UT" / "warp to here"**: not in v1.
- **Uncommitted events**: not shown. They join on commit.